Source Code and Output - Diet Patterns vs BMI

Part A: make_dataset.py

```
"""Generate a SIMULATED lifestyle-survey dataset (same columns as the UCI
'Estimation of obesity levels based on eating habits and physical condition'
dataset).  Used only when the real CSV is not present in data/.
Run:  python make_dataset.py
"""
import numpy as np, pandas as pd

rng = np.random.default_rng(42)
n = 2000

gender = rng.choice(["Male", "Female"], n)
age = np.clip(rng.gamma(6, 4, n) + 14, 14, 61).round(0)
height = np.where(gender == "Male", rng.normal(1.76, 0.07, n), rng.normal(1.63, 0.07, n)).round(2)
family = rng.choice(["yes", "no"], n, p=[0.8, 0.2])
favc = rng.choice(["yes", "no"], n, p=[0.88, 0.12])                      # high-calorie food
fcvc = np.clip(rng.normal(2.4, 0.55, n), 1, 3).round(1)                  # vegetables 1-3
ncp = np.clip(rng.normal(2.7, 0.8, n), 1, 4).round(0)                    # main meals/day
caec = rng.choice(["no", "Sometimes", "Frequently", "Always"], n, p=[.05, .75, .15, .05])
ch2o = np.clip(rng.normal(2.0, 0.6, n), 1, 3).round(1)                   # water 1-3
calc = rng.choice(["no", "Sometimes", "Frequently"], n, p=[.3, .65, .05]) # alcohol
faf = np.clip(rng.gamma(1.6, 0.55, n), 0, 3).round(1)                    # activity 0-3
tue = np.clip(rng.gamma(2.0, 0.4, n), 0, 2).round(1)                     # screen time 0-2
smoke = rng.choice(["no", "yes"], n, p=[.98, .02])
scc = rng.choice(["no", "yes"], n, p=[.95, .05])                         # calorie monitoring
mtrans = rng.choice(["Public_Transportation", "Automobile", "Walking", "Motorbike", "Bike"],
                    n, p=[.58, .22, .15, .03, .02])

snack = pd.Series(caec).map({"no": 0, "Sometimes": 1.0, "Frequently": 2.6, "Always": 3.6}).values
alc = pd.Series(calc).map({"no": 0, "Sometimes": 0.5, "Frequently": 1.5}).values
bmi = (22.0
       + 3.0 * (favc == "yes") - 1.6 * (fcvc - 2.4) + 0.6 * (ncp - 2.7)
       + 1.1 * snack + 0.7 * alc - 0.3 * (ch2o - 2.0)
       - 1.3 * faf + 0.6 * tue + 0.09 * (age - 25)
       + 1.8 * (family == "yes") - 0.8 * (scc == "yes")
       - 1.0 * np.isin(mtrans, ["Walking", "Bike"]) + 0.8 * (mtrans == "Automobile")
       + rng.normal(0, 2.6, n))
bmi = np.clip(bmi, 15, 50)
weight = (bmi * height ** 2).round(1)

pd.DataFrame(dict(Gender=gender, Age=age, Height=height, Weight=weight,
    family_history_with_overweight=family, FAVC=favc, FCVC=fcvc, NCP=ncp, CAEC=caec,
    SMOKE=smoke, CH2O=ch2o, SCC=scc, FAF=faf, TUE=tue, CALC=calc, MTRANS=mtrans)
).to_csv("data/diet_bmi_simulated.csv", index=False)
print("saved data/diet_bmi_simulated.csv", n, "rows")
```

Part B: analysis.py

```
"""Relation between diet patterns and BMI
Run:  python analysis.py
Uses data/ObesityDataSet_raw_and_data_sinthetic.csv (real UCI data) if present,
otherwise data/diet_bmi_simulated.csv (generated by make_dataset.py).
"""
import os, json, warnings
import numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.inspection import permutation_importance
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
os.makedirs("outputs", exist_ok=True)
R = {}  # results saved for the report

# ---------- 1. Load ----------
real = "data/ObesityDataSet_raw_and_data_sinthetic.csv"
path = real if os.path.exists(real) else "data/diet_bmi_simulated.csv"
if not os.path.exists(path):
    os.system("python make_dataset.py")
df = pd.read_csv(path)
R["source"] = "UCI real dataset" if path == real else "Simulated dataset"
print("Data file:", path, "| shape:", df.shape)
print(df.head().to_string()); print("\nMissing values:\n", df.isna().sum().sum())

# ---------- 2. Feature engineering ----------
df["BMI"] = df["Weight"] / df["Height"] ** 2
df["BMI_cat"] = pd.cut(df["BMI"], [0, 18.5, 25, 30, 100],
                       labels=["Underweight", "Normal", "Overweight", "Obese"], right=False)
print("\nBMI summary:\n", df["BMI"].describe().round(2).to_string())
print("\nBMI categories:\n", df["BMI_cat"].value_counts().to_string())
R["n"] = len(df); R["bmi_mean"] = df.BMI.mean(); R["bmi_sd"] = df.BMI.std()
R["cat_pct"] = (df.BMI_cat.value_counts(normalize=True) * 100).round(1).to_dict()

# ---------- 3. EDA plots ----------
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
sns.histplot(df.BMI, bins=40, kde=True, ax=ax[0], color="#2a7f62"); ax[0].set_title("BMI distribution")
for v in (18.5, 25, 30): ax[0].axvline(v, color="grey", ls="--")
df.BMI_cat.value_counts().reindex(["Underweight", "Normal", "Overweight", "Obese"]).plot.bar(ax=ax[1], color="#2a7f62")
ax[1].set_title("BMI categories"); ax[1].set_xlabel("")
plt.tight_layout(); plt.savefig("outputs/01_bmi_distribution.png", dpi=150); plt.close()

diet_cat = ["FAVC", "CAEC", "CALC"]
fig, ax = plt.subplots(1, 3, figsize=(14, 4))
for a, c in zip(ax, diet_cat):
    order = {"CAEC": ["no", "Sometimes", "Frequently", "Always"], "CALC": ["no", "Sometimes", "Frequently"]}.get(c)
    sns.boxplot(data=df, x=c, y="BMI", order=order, ax=a, palette="viridis")
ax[0].set_title("High-calorie food (FAVC)"); ax[1].set_title("Snacking (CAEC)"); ax[2].set_title("Alcohol (CALC)")
plt.tight_layout(); plt.savefig("outputs/02_bmi_by_diet_category.png", dpi=150); plt.close()

num_diet = ["FCVC", "NCP", "CH2O", "FAF", "TUE", "Age"]
fig, ax = plt.subplots(1, 3, figsize=(14, 4))
for a, c in zip(ax, ["FCVC", "NCP", "CH2O"]):
    sns.regplot(data=df.sample(min(len(df), 800), random_state=1), x=c, y="BMI", ax=a,
                scatter_kws={"alpha": .25, "s": 12}, line_kws={"color": "red"})
ax[0].set_title("Vegetable intake"); ax[1].set_title("Main meals / day"); ax[2].set_title("Water intake")
plt.tight_layout(); plt.savefig("outputs/03_bmi_vs_numeric_diet.png", dpi=150); plt.close()

plt.figure(figsize=(6.5, 5))
corr = df[["BMI"] + num_diet].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0)
plt.title("Correlation with BMI"); plt.tight_layout()
plt.savefig("outputs/04_correlation_heatmap.png", dpi=150); plt.close()
print("\nPearson correlation with BMI:\n", corr["BMI"].drop("BMI").round(3).to_string())
R["corr"] = corr["BMI"].drop("BMI").round(3).to_dict()

# ---------- 4. Statistical tests ----------
print("\n=== Hypothesis tests ===")
a = df[df.FAVC == "yes"].BMI; b = df[df.FAVC == "no"].BMI
t, p = stats.ttest_ind(a, b, equal_var=False)
print(f"FAVC yes vs no: mean {a.mean():.2f} vs {b.mean():.2f}, Welch t={t:.2f}, p={p:.2e}")
R["favc"] = dict(yes=a.mean(), no=b.mean(), p=p)
R["anova"] = {}
for c in ["CAEC", "CALC"]:
    groups = [g.BMI.values for _, g in df.groupby(c)]
    H, p = stats.kruskal(*groups)
    means = df.groupby(c).BMI.mean().round(2).to_dict()
    print(f"{c}: Kruskal-Wallis H={H:.1f}, p={p:.2e}, group means={means}")
    R["anova"][c] = dict(p=p, means=means)
chi = stats.chi2_contingency(pd.crosstab(df.CAEC, df.BMI_cat))
print(f"CAEC vs BMI category: chi2={chi[0]:.1f}, p={chi[1]:.2e}")

# ---------- 5. Multiple linear regression (OLS) ----------
print("\n=== Multiple linear regression (BMI ~ diet + controls) ===")
X = pd.get_dummies(df.drop(columns=["BMI", "BMI_cat", "Weight", "Height"]), drop_first=True).astype(float)
y = df.BMI.values
Xs = X.copy()
for c in ["Age", "FCVC", "NCP", "CH2O", "FAF", "TUE"]:
    Xs[c] = (Xs[c] - Xs[c].mean()) / Xs[c].std()
A = np.column_stack([np.ones(len(Xs)), Xs.values])
beta = np.linalg.solve(A.T @ A, A.T @ y)
```

```
res = y - A @ beta
dof = len(y) - A.shape[1]
s2 = res @ res / dof
se = np.sqrt(np.diag(s2 * np.linalg.inv(A.T @ A)))
tv = beta / se; pv = 2 * stats.t.sf(abs(tv), dof)
r2 = 1 - res @ res / ((y - y.mean()) @ (y - y.mean()))
coef = pd.DataFrame({"coef": beta[1:], "se": se[1:], "p": pv[1:]}, index=Xs.columns).sort_values("coef")
print(coef.round(3).to_string()); print(f"\nR-squared = {r2:.3f}  (continuous predictors standardised)")
R["ols_r2"] = r2
R["ols_top"] = coef.reindex(coef.coef.abs().sort_values(ascending=False).index).head(6).round(2)["coef"].to_dict()
plt.figure(figsize=(7, 7))
sig = coef[coef.p < 0.05]
plt.barh(sig.index, sig.coef, xerr=1.96 * sig.se, color=np.where(sig.coef > 0, "#c0392b", "#2a7f62"))
plt.axvline(0, color="k"); plt.title("OLS coefficients (p<0.05, 95% CI)\nChange in BMI units")
plt.tight_layout(); plt.savefig("outputs/05_regression_coefficients.png", dpi=150); plt.close()

# ---------- 6. Random forest + permutation importance ----------
print("\n=== Random Forest regressor ===")
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)
rf = RandomForestRegressor(300, min_samples_leaf=3, random_state=42, n_jobs=-1).fit(Xtr, ytr)
cv = cross_val_score(RandomForestRegressor(200, min_samples_leaf=3, random_state=42, n_jobs=-1), X, y, cv=5, scoring="r2")
print(f"Test R2 = {rf.score(Xte, yte):.3f} | 5-fold CV R2 = {cv.mean():.3f} +/- {cv.std():.3f}")
pi = permutation_importance(rf, Xte, yte, n_repeats=10, random_state=42, n_jobs=-1)
imp = pd.Series(pi.importances_mean, index=X.columns).sort_values(ascending=False)
print("\nTop 10 permutation importances:\n", imp.head(10).round(4).to_string())
R["rf_test_r2"] = rf.score(Xte, yte); R["rf_cv_r2"] = cv.mean()
R["rf_top"] = imp.head(6).round(3).to_dict()
imp.head(12)[::-1].plot.barh(figsize=(7, 5), color="#2a7f62")
plt.title("Random Forest permutation importance"); plt.tight_layout()
plt.savefig("outputs/06_feature_importance.png", dpi=150); plt.close()

json.dump(R, open("outputs/results.json", "w"), indent=2, default=float)
print("\nDone. Figures and results.json saved in outputs/")
```

Part C: Console output (outputs/output_log.txt)

```
Data file: data/diet_bmi_simulated.csv | shape: (2000, 16)
   Gender   Age  Height  Weight family_history_with_overweight FAVC  FCVC  NCP       CAEC SMOKE  CH2O SCC  FAF  TUE        CALC
0    Male  30.0    1.86   108.3                           yes  yes   3.0  3.0  Sometimes    no   2.7  no  0.3  1.1  Frequently  Public_Tr
1  Female  37.0    1.68    85.5                           yes  yes   2.5  3.0  Sometimes    no   1.6  no  1.2  0.3   Sometimes  Public_Tr
2  Female  36.0    1.73    78.2                           yes  yes   2.7  3.0         no    no   1.6  no  0.3  0.5   Sometimes
3    Male  43.0    1.82    96.5                           yes  yes   2.6  3.0  Sometimes    no   2.5  no  0.2  1.8   Sometimes
4    Male  28.0    1.84    73.7                            no  yes   3.0  3.0  Sometimes    no   1.8  no  0.3  2.0   Sometimes

Missing values:
 0

BMI summary:
 count    2000.00
mean       28.26
std         3.44
min        15.18
25%        25.98
50%        28.36
75%        30.60
max        41.23

BMI categories:
 BMI_cat
Overweight     1053
Obese           610
Normal          331
Underweight       6

Pearson correlation with BMI:
 FCVC   -0.238
NCP     0.169
CH2O   -0.035
FAF    -0.242
TUE     0.106
Age     0.246

=== Hypothesis tests ===
FAVC yes vs no: mean 28.57 vs 25.51, Welch t=12.02, p=1.76e-26
CAEC: Kruskal-Wallis H=99.5, p=1.96e-21, group means={'Always': 31.14, 'Frequently': 29.27, 'Sometimes': 27.95, 'no': 27.37}
CALC: Kruskal-Wallis H=16.3, p=2.90e-04, group means={'Frequently': 29.77, 'Sometimes': 28.24, 'no': 28.11}
CAEC vs BMI category: chi2=100.2, p=1.41e-17

=== Multiple linear regression (BMI ~ diet + controls) ===
                                  coef     se      p
CAEC_no                         -3.437  0.388  0.000
CAEC_Sometimes                  -2.988  0.294  0.000
MTRANS_Walking                  -1.782  0.196  0.000
CALC_no                         -1.541  0.323  0.000
CAEC_Frequently                 -1.432  0.324  0.000
CALC_Sometimes                  -1.419  0.312  0.000
MTRANS_Bike                     -1.045  0.440  0.018
FAF                             -0.829  0.060  0.000
FCVC                            -0.800  0.060  0.000
MTRANS_Motorbike                -0.781  0.379  0.040
MTRANS_Public_Transportation    -0.760  0.150  0.000
SCC_yes                         -0.719  0.298  0.016
CH2O                            -0.171  0.060  0.004
SMOKE_yes                       -0.103  0.426  0.808
Gender_Male                      0.290  0.119  0.015
TUE                              0.344  0.060  0.000
NCP                              0.527  0.060  0.000
Age                              0.825  0.060  0.000
family_history_with_overweight_yes  1.697  0.149  0.000
FAVC_yes                         3.167  0.198  0.000

R-squared = 0.411  (continuous predictors standardised)

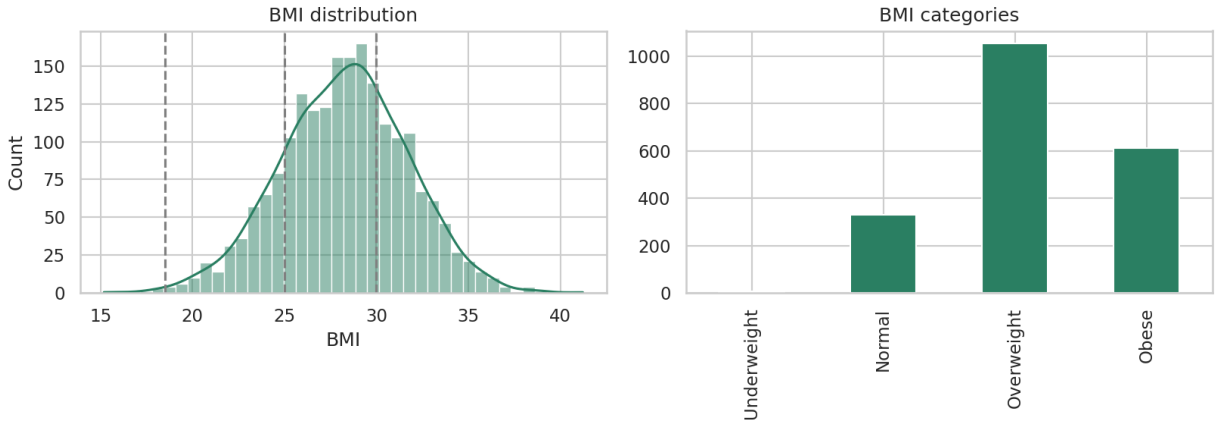
=== Random Forest regressor ===
Test R2 = 0.281 | 5-fold CV R2 = 0.288 +/- 0.035

Top 10 permutation importances:
 Age                                0.1184
FCVC                               0.1107
FAVC_yes                           0.1030
FAF                                0.0777
family_history_with_overweight_yes 0.0538
CAEC_Sometimes                     0.0258
MTRANS_Walking                     0.0242
NCP                                0.0162
TUE                                0.0136
CAEC_no                            0.0118

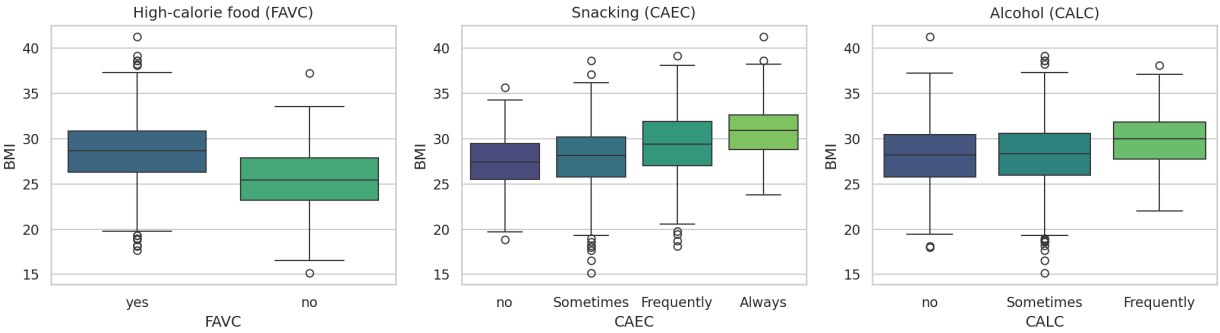
Done. Figures and results.json saved in outputs/
```

Part D: Figures

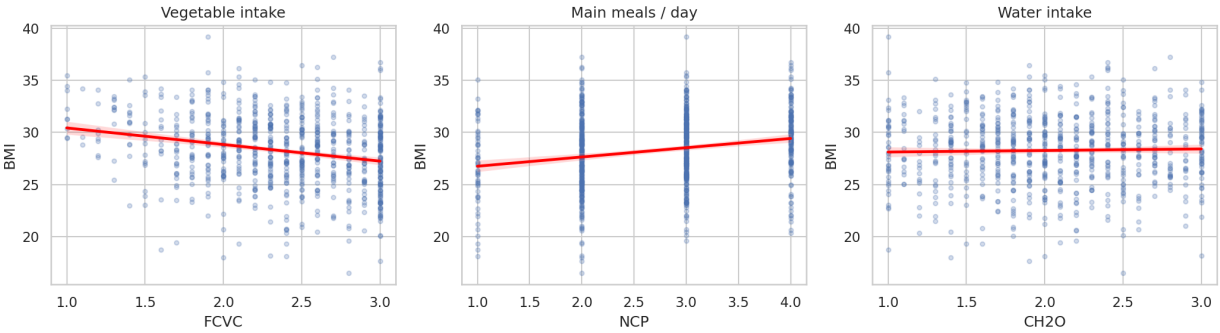
01_bmi_distribution.png



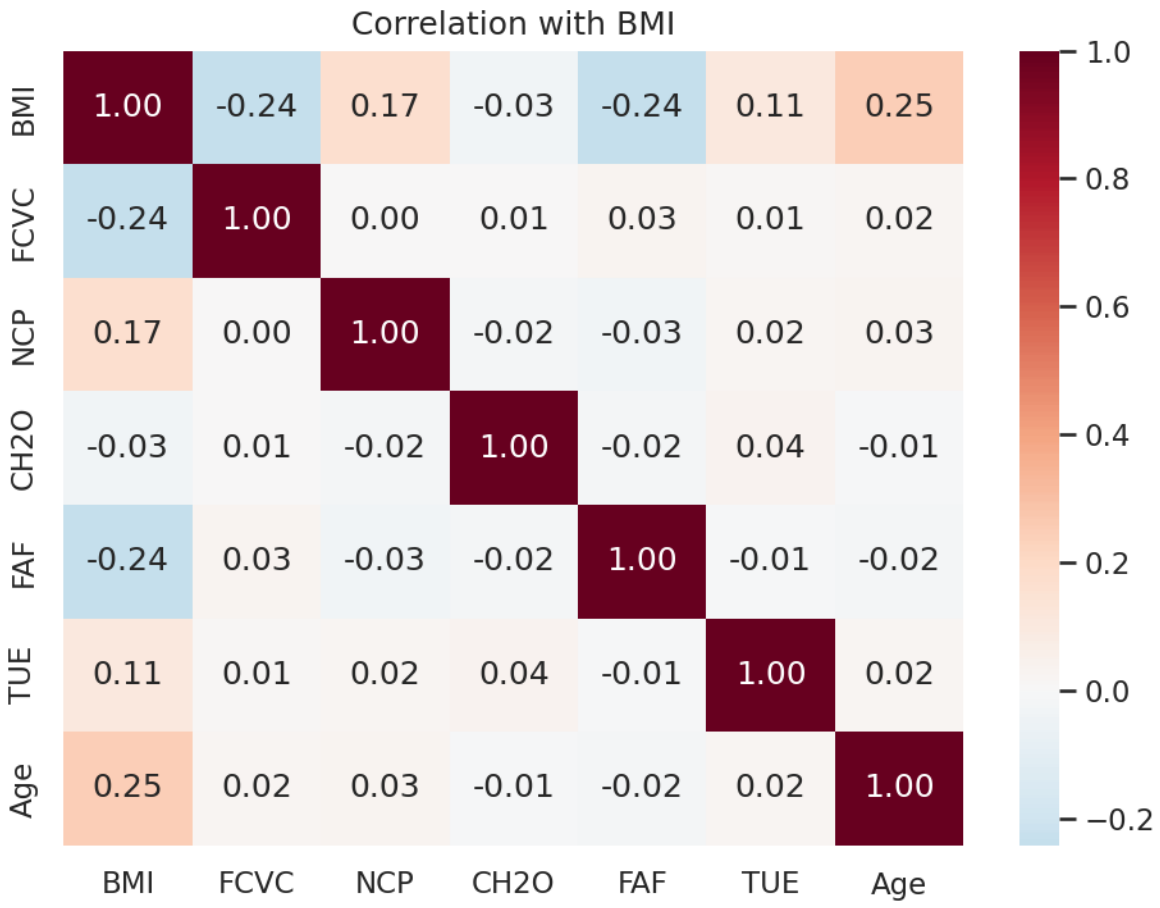
02_bmi_by_diet_category.png



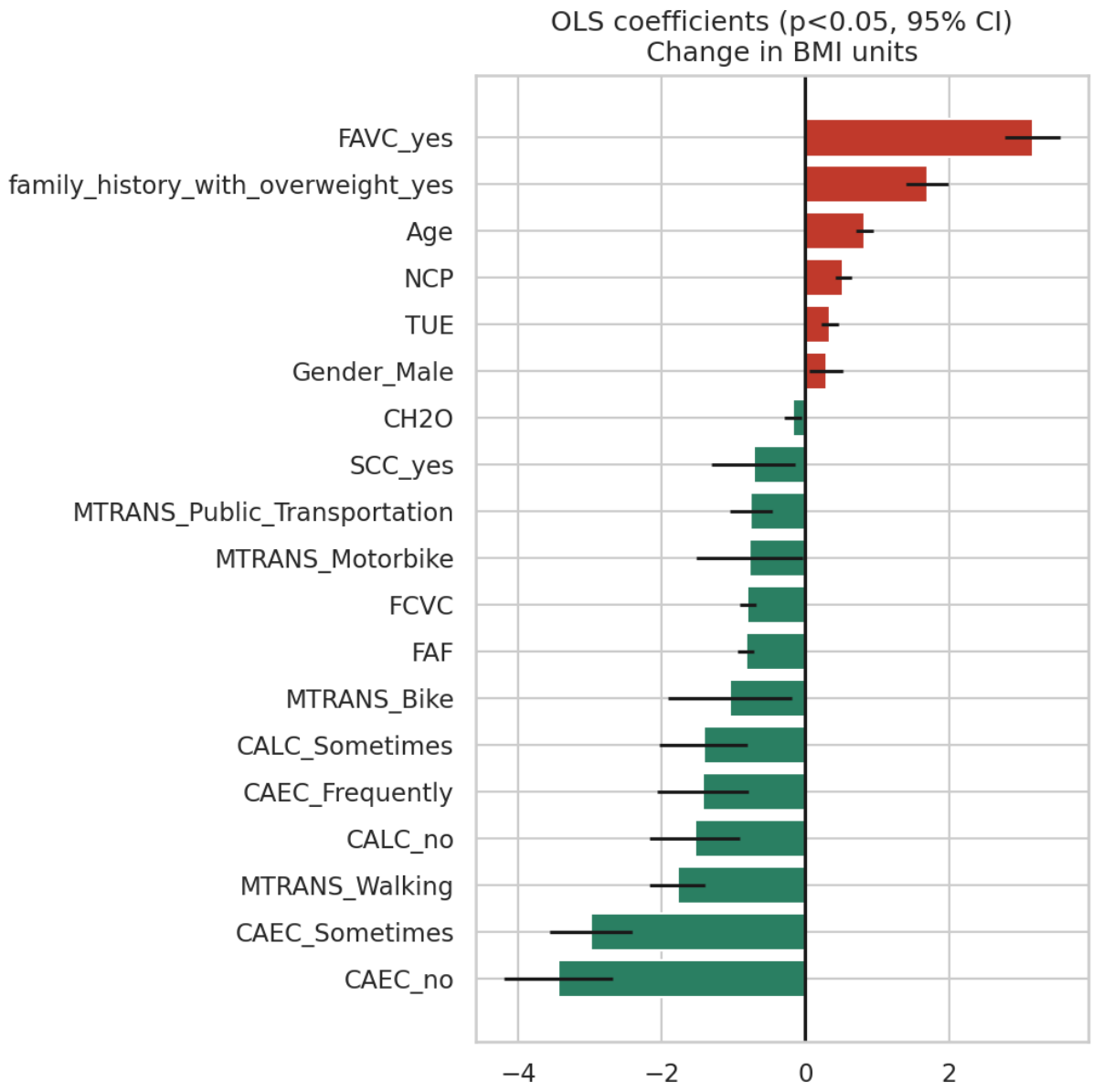
03_bmi_vs_numeric_diet.png



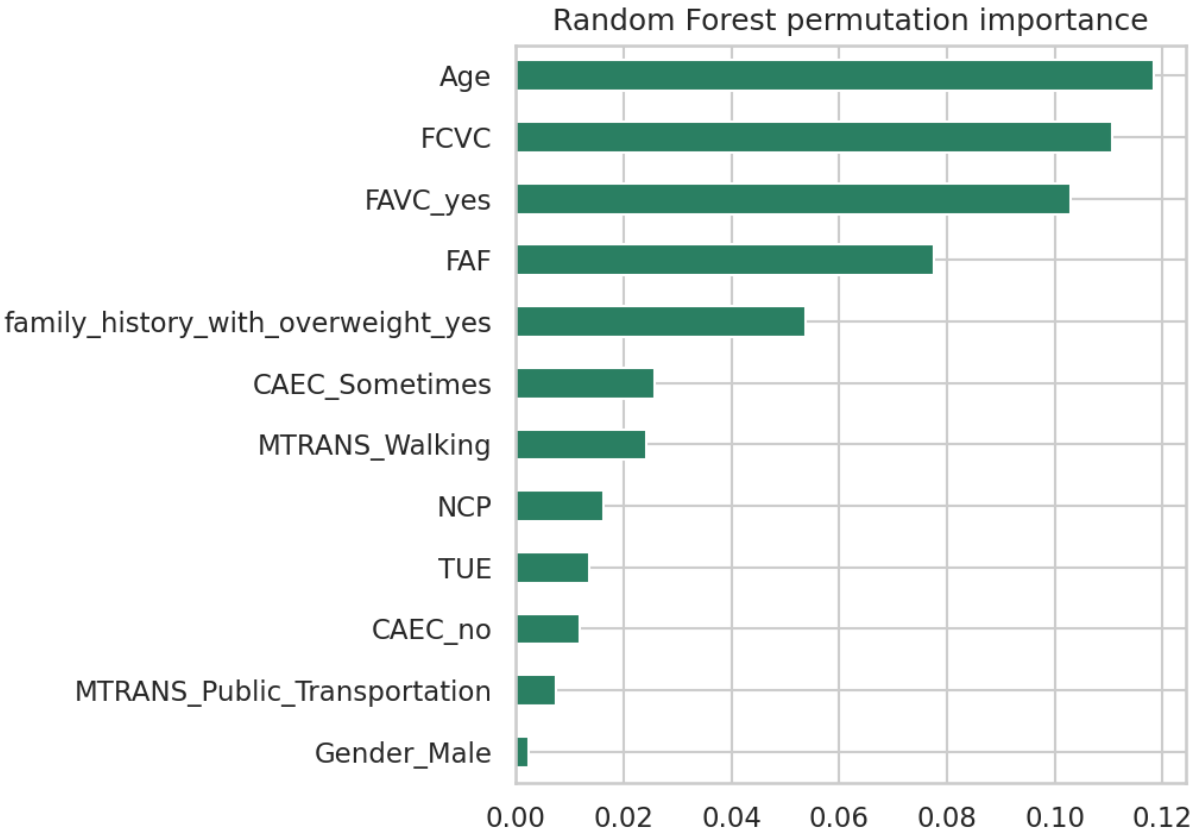
04_correlation_heatmap.png



05_regression_coefficients.png



06_feature_importance.png


Random Forest permutation importance
Age
FCVC
FAVC_yes
FAF
family_history_with_overweight_yes
CAEC_Sometimes
MTRANS_Walking
NCP
TUE
CAEC_no
MTRANS_Public_Transportation
Gender_Male
0.00
0.02
0.04
0.06
0.08
0.10
0.12